

SSBI: Technische und mathematische CellCNN-Analyse

Baseline, GrHa und GrHa-learnable-pooling

9. September 2026

Inhaltsverzeichnis

1. Executive Summary	2
2. Baseline CellCNN	3
3. Phenotype/Interpretation der Baseline	6
4. GrHa-Architektur	7
5. Geometrie der Mahalanobis-/Prototype-Filter	8
6. GrHa-Resultate	9
7. Learnable-Pooling-Architektur	10
8. Direkter Codevergleich GrHa versus Learnable Pooling	11
9. Warum die Performance sinkt	12
10. Phenotype-Scoring und Clustering	13
11. Datenmenge, CV und Refit	15
12. ReLU-Radius versus Alpha	17
13. Regularisierung	17
14. Konkrete Empfehlungen	18
15. Kleine kontrollierte Ablationsstudie	19
Konkrete nächste Schritte	21
Appendix: Code- und Quellenreferenzen	21

Stand: 9. September 2026. Grundlage sind geprüfter Quellcode, gespeicherte Ergebnisse und lokale Originaldaten. **[CODE]** bezeichnet Implementierungsbefunde, **[RESULT]** nachgerechnete Resultate, **[INTERPRETATION]** deren Einordnung und **[RECOMMENDATION]** vorgeschlagene Änderungen. Diese ausführliche Analyse ist ein Begleitdokument zum fünfseitigen Gruppenbericht.

1. Executive Summary

Die lineare Baseline ist im vorhandenen 100-Split-Vergleich besser als beide untersuchten Prototypvarianten. Der Performanceverlust kann nicht allein dem lernbaren Alpha zugeschrieben werden.

Verfahren und belegter Lauf	Splits	Mittlere Network-AUC	Baseline auf denselben Splits
Lineare CellCNN-Baseline	100	0,80750	0,80750
GrHa: Prototyp + ReLU-Radius + Mean	100	0,63625	0,80750
GrHa: Prototyp + ReLU-Radius + Top1	3	0,70833	1,00000
GrHa: linear-quadratisch + ReLU + Top1	10	0,93750	0,87500
Learnable Pooling v1, ohne Geometriestrafe	100	0,71875	0,80750
Learnable Pooling v2, mit Geometriestrafe	50	0,72250	0,80250

[RESULT] Diese Werte wurden aus den gespeicherten Donorvorhersagen nachgerechnet. Die unterschiedlichen Splitumfänge erlauben keine unmittelbare Rangliste anhand der unbereinigten Mittelwerte. Quellen: R100, RM, RT, RQ, R50.

Die wichtigsten Korrekturen deiner Ausgangsannahmen:

- **GrHa ist nicht ein einziges Prototypmodell.** 06b verwendet Mean-Pooling über alle Zellen, 06c Top-1%-Pooling. Der besonders gute Wert stammt aus 06d, einem anderen, linear-quadratischen Modell.
- **Learnable Pooling ist nicht einfach „GrHa plus Alpha“.** Radius und ReLU entfallen; die Zellantwort wird negativ, das Pooling und die Interpretationsmetrik ändern sich. Gegenüber 06b ändert sich zusätzlich die Aggregation von allen Zellen auf einen weichen Rangbereich.
- **„Nur Output-L2“ gilt für v1, nicht vollständig für den aktuellen Branch.** Seit Commit 1d696d6 wird zusätzlich die Abweichung normalisierter Markergewichte von 1 bestraft. Auf denselben 50 Splits steigt die AUC gegenüber v1 lediglich von 0,7200 auf 0,7225.
- **Keines dieser Netzverfahren refittet das ausgewählte Modell auf allen 14 Outer-Train-Donoren.** Es wird ein auf neun oder zehn Donoren trainiertes Inner-Fold-Modell getestet. Das folgt dem NK-Benchmark des Papers; ein Refit wäre eine begründete Erweiterung.
- **Netzwerk, Population und Clustering sind getrennte Schritte.** Das Netzwerk clustert nicht. Die neuere lokale Interpretation gruppiert bereits Populationszentroiden; das ist kein Clustering der einzelnen ausgewählten Zellen.

[CODE] Versionsabgrenzung: GitHub-GrHa steht auf 4477d6b, der lokale Branch vier Commits weiter auf 68452ce. GitHub-GrHa-learnable-pooling steht auf 1d696d6; dessen gespeicherter 100-Split-Lauf stammt aus 544192d. Die Baseline in 04c ist in beiden GitHub-Branches identisch. Der aktuelle main enthält diese CellCNN-Notebooks nicht. Vollständige Referenzen stehen im Appendix.

[RECOMMENDATION] Ich wähle Option B: kleine, kontrollierte Änderungen. Zuerst Alpha mit derselben weichen Maske fixieren und Radius/Pooling getrennt untersuchen; danach einen sauberen Refit vergleichen. Für biologische Aussagen braucht das Prototypmodell zusätzlich eine eingefrorene absolute Populationsgrenze. Eine größere Architektur oder weitere Regularisierungsparameter haben derzeit geringere Priorität.

2. Baseline CellCNN

Daten, Transformation und Skalierung

[CODE] 04c, Codezellen 4, 6, 8 und 10: Die Full-Pipeline liest `gated_alive`, nicht ausschließlich NK-Zellen. Sie nutzt 3.438.750 Ereignisse aus 20 FCS-Dateien und 37 Marker. `gated_NK` mit 261.593 Ereignissen dient dem Smoke-Modus. Die Marker werden anhand ihrer FCS-Kurznamen in der Reihenfolge der Markerdatei ausgewählt. Technische Kanäle wie `Time`, `Dead` und DNA-Kanäle sind nicht Bestandteil dieser 37 Merkmale. B

Das Gating wird nicht neu gelernt: Der Code lädt die bereits bereitgestellten Gate-Dateien. Die offizielle ungated-NK-Referenz beschreibt diese breitere Eingabe als Daten nach Entfernung toter Zellen und Doubletten. O-U

Ohne Umgebungsüberschreibung startet 04c im Smoke-Modus. Die folgenden Trainingsparameter und berichteten Benchmark-AUCs gehören zum expliziten Full-Modus mit `TASK4_RUN_MODE=full`.

Die Labeldatei enthält **11 Donoren mit Label 0 und neun mit Label 1**. Die offiziellen NK-Notebooks bezeichnen diese explizit als CMV[−] beziehungsweise CMV⁺: Serostatus bezüglich einer früheren CMV-Infektion, keine direkte Zelltypannotation und keine Diagnose einer aktuell aktiven Infektion. O-NK, O-U

Sei r_{inj} der rohe Messwert von Marker j in Zelle n des Donors i . Zunächst gilt

$$u_{inj} = \operatorname{arsinh}(r_{inj}/5), \quad x_{inj} = \frac{u_{inj} - \mu_j}{\sigma_j}.$$

`load_transformed_fcs()` liest `source=rrow`; der Code führt anschließend selbst die `arsinh`-Transformation aus. `fit_balanced_scaler()` zieht ohne Zurücklegen 20.000 Zellen **je tatsächlichem inneren Trainingsdonor**. `StandardScaler` schätzt auf deren Vereinigung Mittelwert und Populationsvarianz. Jeder Trainingsdonor trägt gleich viele Zellen zur Skalierung bei. Ein Marker ohne Varianz erhielte gemäß `StandardScaler` Skalierungsfaktor 1.

Validierungs- und Testdonoren werden ausschließlich mit diesem Scaler transformiert. Derselbe eingefrorene Scaler gehört später zum ausgewählten Modell. Dass Rohdaten bereits geladen sind, ist kein Leakage: Entscheidend ist, welche Donoren in die Schätzung eingehen. Die Projektionen aus Aufgabe 2 werden nicht als Klassifikatoreingabe verwendet.

[RESULT] Alle 23 gespeicherten Eingangshashes – 20 FCS-Dateien, Marker, Labels und Splits – stimmen mit den vorhandenen Dateien überein. Die spenderbalancierte Skalierung des ausgewählten Baseline-Modells für Split 0 wurde aus den Originaldaten bis auf 10^{-12} reproduziert.

Bags und unabhängige Stichprobengröße

[CODE] `materialize_multicell_inputs()` erzeugt im Full-Modus **200 feste Bags je Donor mit jeweils 3.000 Zellen**, gezogen mit Zurücklegen. Jeder Bag erhält das Donorlabel. Diese Bags werden einmal materialisiert und über die Epochen wiederverwendet; pro Epoche wird lediglich ihre Reihenfolge im `DataLoader` gemischt. Auch die Validierungs-Bags haben diese Größe und Anzahl. B

Bei neun beziehungsweise zehn Trainingsdonoren entstehen somit 1.800 beziehungsweise 2.000 Trainings-Bags. Ihre Zahl ist nicht die biologische Stichprobengröße. Bedingt auf einen festen Donorzellpool können getrennte Zufallsziehungen unabhängig sein; für die Generalisierung auf neue Menschen teilen sie jedoch Donoreigenschaften und dasselbe Label. **Die unabhängige Beobachtungseinheit der Projektanalyse ist der Donor: insgesamt 20, je ausgewähltem Netz neun oder zehn im Gradienten-Training.** Die weiteren vier oder fünf Outer-Train-Donoren beeinflussen die Auswahl und den Trainingsstopp.

Zur Vorhersage zieht `predict_donor()` fünf Bags mit jeweils bis zu 20.000 Zellen **ohne Zurücklegen innerhalb eines Bags**. Zwischen den fünf Bags können Zellen erneut vorkommen. Gleich viele Bags gewichten Donoren gleich; eine zusätzliche Klassengewichtung der Cross Entropy existiert nicht.

Zellfilter, Geometrie und Pooling

Für Filter k implementiert `CellCNN.forward()` exakt

$$z_k(x) = w_k^\top x + b_k, \quad h_k(x) = \max(0, z_k(x)).$$

w_k ist die Normale einer Hyperebene im standardisierten Markerraum. Die positive Region ist der Halbraum $w_k^\top x + b_k > 0$. Positive Gewichte belohnen relativ hohe Markerwerte, negative Gewichte relativ niedrige. Mehrere Marker können einander kompensieren: Das ist kein System unabhängiger Marker-Gates und insbesondere keine zwingende logische UND-Verknüpfung.

Die ReLU setzt die Antwort außerhalb des Halbraums auf null. Innerhalb steigt sie linear mit dem Abstand zur Grenzebene, skaliert mit $\|w_k\|$. Der Gewichtsvektor ist deshalb weder ein Zellprofil noch der Mittelwert der ausgewählten Population.

Für einen Bag B mit N Zellen werden die Antworten je Filter absteigend sortiert. Der Code setzt

$$K_N = \max(1, \lfloor 0,01N \rfloor), \quad P_k(B) = \frac{1}{K_N} \sum_{r=1}^{K_N} h_{(r)k}.$$

Es handelt sich um **Top-k-Mittelung**, nicht um einen einzelnen Maximalwert und nicht um ein Perzentil als absolute Schwelle. Im Trainings-Bag werden 30, im 20.000-Zellen-Vorhersage-Bag 200 Antworten gemittelt. Sortierung und Pooling erfolgen für jeden Filter separat.

Für seltene Populationen verhindert dies, dass ihre Antwort über sämtliche Hintergrundzellen verdünnt wird. Liegt eine Population mit Anteil $f < 0,01$ vor, reagiert sie ungefähr mit Höhe a und der Hintergrund mit null, ergibt sich näherungsweise $P_k \approx af/0,01$ statt af beim Gesamtmittel. Bei $f > 0,01$ kann Top1 dagegen die stärksten Antworten auswählen, ohne die gesamte Populationshäufigkeit abzubilden. Top-Pooling misst daher keine reine Frequenz.

Output, Score und Loss

Mit Output-Gewichten $V \in \mathbb{R}^{2 \times K}$ und Bias β lauten die beiden Bag-Logits

$$\ell_c(B) = \beta_c + \sum_{k=1}^K V_{ck} P_k(B), \quad p_1(B) = \frac{e^{\ell_1(B)}}{e^{\ell_0(B)} + e^{\ell_1(B)}}.$$

Im binären Fall ist das zugleich

$$p_1(B) = \text{sigmoid}\left(\beta_1 - \beta_0 + \sum_k \Delta_k P_k(B)\right), \quad \Delta_k = V_{1k} - V_{0k}.$$

Der gespeicherte Donorscore ist

$$\hat{p}_i = \frac{1}{5} \sum_{b=1}^5 p_1(B_{ib}), \quad \hat{y}_i = \mathbf{1}\{\hat{p}_i \geq 0,5\}.$$

Es werden **Wahrscheinlichkeiten gemittelt**, nicht Logits. Wegen der Nichtlinearität ist das im Allgemeinen nicht dasselbe. Die Network-ROC-AUC verwendet $\hat{p}_i$ und die Donorlabels.

Die tatsächlich pro Minibatch $\mathcal{B}$ berechnete Trainingsloss lautet

$$\mathcal{L}_{\mathcal{B}} = -\frac{1}{|\mathcal{B}|} \sum_{b \in \mathcal{B}} \log \text{softmax}(\ell(B_b))_{y_b} + 10^{-4} \left(\|W\|_F^2 + \|V\|_F^2 \right).$$

[CODE] Regularisiert werden `cell_filters.weight` und `output_layer.weight`. Beide Biasvektoren bleiben unreguliert. Es gibt kein Dropout und keinen zusätzlichen Optimizer-Weight-Decay. Adam verwendet Lernrate 0,01 und die Standardwerte der übrigen Argumente. `nn.Linear` initialisiert Gewichte und Bias standardmäßig gleichverteilt in $[-1/\sqrt{d_{\text{in}}}, 1/\sqrt{d_{\text{in}}}]$; die Seedsetzung macht das reproduzierbar. B

Die Validierungs-Loss ist dieselbe regulierte Bag-Loss, korrekt nach Baganzahl gemittelt. Sie ist keine Donor-AUC. Die protokollierte Trainingsloss ist hingegen der ungewichtete Mittelwert der Batch-Losses; bei einem kürzeren letzten Batch entspricht sie nicht exakt dem Mittel über sämtliche Bags.

Modellselektion

[CODE] 04a erstellt mit Master-Seed 12345 100 wiederholte zufällige, spenderweise Splits. Jeder enthält 14 Outer-Train-Donoren, sieben je Klasse, und sechs Testdonoren, davon vier CMV- und zwei CMV+. Innerhalb der 14 Trainingsdonoren erzeugt `StratifiedKFold(3, shuffle=True, random_state=split_seed)` Validierungsgruppen der Größen 5, 5 und 4. S

`train_outer_split()` trainiert für jeden Inner-Fold die Filterzahlen 3, 4 und 5: **neun Kandidaten pro Full-Split**. Der Kandidatenseed ist

$$s_{\text{candidate}} = s_{\text{split}} + 10\,000 f + 100 K.$$

Batchgröße ist 128, maximal werden 100 Epochen trainiert. Eine Verbesserung der Validierungs-Loss muss größer als 10^{-6} sein. Nach fünf Epochen ohne solche Verbesserung wird gestoppt; anschließend wird der Zustand mit der besten Validierungs-Loss wiederhergestellt.

Die neun **einzelnen Modelle** werden danach lexikographisch sortiert:

1. höhere Donor-Validierungs-Accuracy bei Schwelle 0,5;
2. höhere Donor-Validierungs-AUC;
3. kleinere beste regulierte Validierungs-Loss;
4. kleinere Filterzahl;
5. kleinerer Inner-Fold-Index.

Es wird keine mittlere Leistung einer Filterzahl über alle drei Folds zur Auswahl benutzt. Das beste Einzelmodell wird direkt auf Outer-Test ausgewertet, **ohne Refit**. Sein Gradienten-Training umfasste neun oder zehn Donoren. Die 14 Outer-Train-Donoren wurden somit für Training und Auswahl verwendet, aber nicht gemeinsam zum finalen Fit.

Die offizielle NK-Benchmarkbeschreibung des Papers verwendet ebenfalls das beste Inner-CV-Netz für den Test. Nicht-Refitting ist hier also papernah. Bewusste Projektabweichungen sind insbesondere das kleine feste Kandidatengitter, die spenderbalancierte Scalerstichprobe und fünf Vorhersage-Bags statt eines Testinputs. Die offiziellen Beispielnotebooks besitzen wiederum andere konkrete Einstellungen und sind nicht identisch mit dem vollständigen Paperbenchmark. P, O-NK, O-U

Zelle: 37 Rohmarker

- > `arcsinh(x/5)`
- > Scaler des tatsächlichen Inner-Train-Sets
- > linearer Filter + ReLU
- > je Filter Mittel der stärksten 1 % im Bag
- > linearer Output -> zwei Logits -> Softmax
- > Training: Bag-Cross-Entropy + L2
- > Vorhersage: Mittel aus fünf Bag-Wahrscheinlichkeiten
- > Donorscore -> Donor-AUC / Klasse bei 0,5

3. Phenotype/Interpretation der Baseline

Was 06a tatsächlich misst

[CODE] `strongest_positive_filter()` definiert

$$k^* = \arg \max_{k: \Delta_k > 0} \Delta_k.$$

Existiert kein positiver Kontrast, liefert die Funktion `None`; die Phänotypmetriken bleiben fehlend. „Stärkster“ bedeutet also **größter positiver Output-Gewichtskontrast**, nicht höchste Antwort, größte Population oder größter tatsächlicher Beitrag zum Donorscore. F

Δ_k beschreibt den Einfluss einer Einheit gepoolter Filterantwort auf den Logitkontrast. Ohne Berücksichtigung der Antwortskala ist es kein vollständiges Wichtigkeitsmaß. Beispielsweise kann man bei der ReLU einen Filter mit einem positiven Faktor skalieren und seinen Output-Kontrast invers skalieren, ohne die Vorhersage zu ändern; die Rangfolge der Kontraste kann sich dabei ändern.

Für das ausgewählte Filter bestimmt 06a auf **allen Zellen seiner tatsächlichen inneren Trainingsdonoren**

$$M_k = \max_{i \in T_{\text{inner}}} \max_n h_k(x_{in}), \quad t_k = \frac{1}{2} M_k.$$

Die Schwelle verwendet weder Inner-Validierungs- noch Outer-Test-Zellen. Auf einem festen, modellübergreifend gleichen Sample E_i von bis zu 20.000 Originalzellen je Testdonor wird dann

$$C_{ik} = \{n \in E_i : h_k(x_{in}) > t_k\}, \quad \hat{f}_{ik} = \frac{|C_{ik}|}{|E_i|}$$

berechnet. Es ist eine Stichprobenschätzung der Donorhäufigkeit, nicht zwingend der exakte Anteil an allen FCS-Ereignissen. Die Auswahl verwendet Seed 63000 + Donorindex und Ziehen ohne Zurücklegen. Bei $M_k = 0$ setzt der Code die Häufigkeit auf null.

Für einen Score g_i definiert sich die empirische AUC als

$$\text{AUC}(g) = \frac{1}{n_+ n_-} \sum_{i: y_i=1} \sum_{j: y_j=0} \left[\mathbf{1}\{g_i > g_j\} + \frac{1}{2} \mathbf{1}\{g_i = g_j\} \right].$$

`frequency_auc` ist diese AUC mit $g_i = \hat{f}_{ik^*}$. Der Code dreht die Richtung nicht nachträglich um. `frequency_effect` ist

$$\bar{f}_+ - \bar{f}_- = \frac{1}{n_+} \sum_{y_i=1} \hat{f}_{ik^*} - \frac{1}{n_-} \sum_{y_i=0} \hat{f}_{ik^*}.$$

Das ist eine **unstandardisierte Häufigkeitsdifferenz**, kein Cohen- d . Ein Wert von 0,0074 entspricht ungefähr 0,74 Prozentpunkten. Ein positiver Output-Kontrast garantiert keinen positiven Testeffekt: Er beschreibt die Netzfunktion, nicht deren erfolgreiche Generalisierung.

05 und Clustering sind versionsabhängig

[CODE] Im GitHub-Stand führt `src/task5_interpretation.py::cellcnn_masks()` eine andere Zusammenfassung als 06a aus: Es vereinigt die Halbmaximum-Selektionen **aller** Filter mit positivem beziehungsweise negativem Kontrast. Eine Zelle kann in beide Vereinigungen fallen. Die anschließenden Zell-Scores aggregieren Selektionen über die äußeren Testauftritte. Das ist nicht die Häufigkeits-AUC eines einzigen stärksten positiven Filters. Ein Clustering dieser ausgewählten Einzelzellen findet dort nicht statt. I

Im neueren lokalen Stand 68452ce erzeugt `halfmax_centroids()` dagegen für jeden wirksamen Filter einen Zentroiden seiner ausgewählten Population. Referenz sind die ursprünglichen 20.000 Scaler-Zellen je tatsächlichem Trainingsdonor, nicht sämtliche Trainingszellen wie in 06a. `group_centroids()` gruppiert diese Zentroiden mit Average-Linkage, Kosinusdistanz und Cutoff 0,4. Verwendet werden 30 Splits; Gruppen ab sechs vorkommenden Splits werden behalten. IL

[RESULT] Die lokale Tabelle weist für die häufigste CellCNN-Gruppe 58 Zentroiden aus allen 30 Splits aus. Das beschreibt Wiederkehr innerhalb dieses Datensatzes. Die globale explorative Skalierung für diese Gruppierung gehört zur retrospektiven Darstellung und fließt nicht in die Klassifikation ein. IG

Damit gilt: **Das Netzwerk clustert nicht; die lokale Interpretation clustert bereits Populationszentroiden; ein zusätzliches Clustering der stark reagierenden Einzelzellen ist damit noch nicht durchgeführt.** Auch das Paper verwendet im NK-Benchmark Zentroid-Gruppierung und beschreibt für andere Interpretationssituationen zusätzlich DBSCAN auf ausgewählten Zellen. Die Behauptung „das ursprüngliche CellCNN hat überhaupt kein Clustering“ wäre daher für das gesamte Paper zu weitgehend. P

[INTERPRETATION] Die Aussage des Professors lässt sich fachlich so verstehen: Zuerst für reale Zellen $h_k(x)$ berechnen, dann nach dieser Antwort selektieren und die Markerprofile der selektierten Zellen untersuchen. w_k beschreibt eine Entscheidungsvorschrift, keine beobachtete Zellpopulation. Gewichte dürfen zur Erklärung der Vorschrift verwendet werden, ersetzen aber nicht die Antwortberechnung und die biologische Charakterisierung ausgewählter Zellen. Seine genaue beabsichtigte Aussage ist aus dem Repository nicht nachweisbar.

4. GrHa-Architektur

[CODE] Der direkte Vergleich lautet: B, M, T

$$\begin{aligned} \text{Baseline: } h_k(x) &= \text{ReLU}(w_k^\top x + b_k), & P_k &= \text{TopMean}_{1\%}(h_k); \\ \text{GrHa 06b: } h_k(x) &= \text{ReLU}(\rho_k - d_k^2(x)), & P_k &= \text{Mean}_{\text{alle}}(h_k); \\ \text{GrHa 06c: } h_k(x) &= \text{ReLU}(\rho_k - d_k^2(x)), & P_k &= \text{TopMean}_{1\%}(h_k). \end{aligned}$$

Output-Layer, Baggrößen, Filterzahlen, Lernrate und Auswahlverfahren entsprechen dem Baseline-Gerüst. Die Prototypmodelle bestrafen ursprünglich nur die Output-Gewichtsmatrix mit L2.

`initialize_prototypes()` zieht zunächst bis zu 2.000 standardisierte Zellen je Trainingsdonor ohne Zurücklegen. Aus der spenderbalancierten Vereinigung werden drei bis fünf Zellen als initiale Zentren gewählt. Ein eigener NumPy-Generator mit `candidate_seed +300000` verändert die Bagziehungen nicht. `raw_a` beginnt bei null, entsprechend uniformen normalisierten Markergewichten. Der initiale Radius jedes Filters ist das 1%-Quantil seiner Distanzen in dieser Trainingsreferenz. Die inverse Softplus setzt den zugehörigen Rohparameter.

Numerisch setzt der Code genauer $\rho_k^{(0)} = \max(Q_{0,01}(d_k^2), 2\varepsilon)$ und $\text{raw_rho}_k = \text{softplus}^{-1}(\rho_k^{(0)} - \varepsilon)$. Dabei ist $\text{softplus}(u) = \log(1 + e^u)$; der kleine positive Offset verhindert eine exakt verschwindende Distanzgewichtung beziehungsweise Radiusgröße.

Zusätzliche Variante 06d:

$$h_k(x) = \text{ReLU} \left(b_k + w_k^\top x + \sum_j q_{kj} x_j^2 \right), \quad P_k = \text{TopMean}_{1\%}(h_k).$$

Die quadratischen Koeffizienten sind unbeschränkt; `quadratic_weights` startet bei null. Dadurch startet das Netz mit derselben Funktion und Zufallsinitialisierung wie die lineare Baseline. L2 wirkt auf lineare, quadratische und Output-Gewichte. Dieses Modell ist **kein positives Distanzmodell**: Es kann je nach Vorzeichen unter anderem nach innen oder außen gerichtete quadratische Regionen erzeugen. Kreuzterme fehlen auch hier. Q

5. Geometrie der Mahalanobis-/Prototype-Filter

[CODE] Mit $d = 37$, $\varepsilon = 10^{-6}$ und Rohparametern r_{kj} gilt

$$v_{kj} = \text{softplus}(r_{kj}) + \varepsilon, \quad a_{kj} = \frac{v_{kj}}{d^{-1} \sum_l v_{kl}}, \quad \frac{1}{d} \sum_j a_{kj} = 1.$$

Alle a_{kj} sind positiv. Die Normalisierung fixiert die mittlere Markergewichtung und verhindert, dass eine gemeinsame Skalierung sämtlicher Gewichte einfach die gesamte Distanzskala ändert. Das trainierbare Zentrum c_k ist ein Punkt im **standardisierten arcsinh-Markerraum**.

`distances()` berechnet algebraisch

$$d_k^2(x) = \frac{1}{d} \sum_{j=1}^d a_{kj} (x_j - c_{kj})^2 = \frac{\sum_j a_{kj} x_j^2 - 2 \sum_j a_{kj} c_{kj} x_j + \sum_j a_{kj} c_{kj}^2}{d}.$$

Die zweite Form wird mit Matrixmultiplikationen ausgewertet. Anschließend setzt `clamp_min(0)` kleine numerisch negative Werte auf null. Mathematisch ist der Ausdruck ohnehin nichtnegativ. Es wird **keine Quadratwurzel** gezogen. M, L2

In der Notation $(x - c)^\top P(x - c)$ ist hier

$$P_k = \frac{1}{d} \text{diag}(a_{k1}, \dots, a_{kd}).$$

Das ist eine **diagonale Mahalanobis-artige quadratische Distanz**, keine vollständige gelernte Präzisionsmatrix und keine explizit geschätzte inverse Kovarianz. Allgemeine Marker-Korrelationen werden nicht durch Off-Diagonaleinträge modelliert.

Die Region $d_k^2(x) < \rho_k$ lässt sich schreiben als

$$\sum_j \frac{(x_j - c_{kj})^2}{d\rho_k/a_{kj}} < 1.$$

Sie ist ein Ellipsoid mit Halbachsen $\sqrt{d\rho_k/a_{kj}}$. Größeres a_{kj} bedeutet eine engere Toleranz gegenüber Abweichungen in Marker j . Quadratische Terme erzeugen also sehr wohl nichtlineare Grenzen: Bereits $(x_1 - c_1)^2 + (x_2 - c_2)^2 < r^2$ beschreibt eine Kreisscheibe.

Matrixstruktur	Geometrie in den Modellkoordinaten
$P = \lambda I, \lambda > 0$	Kugel; in zwei Dimensionen Kreis
Positives diagonales P	Achsenparalleles Ellipsoid
Vollständiges positiv definites P	Allgemein rotiertes Ellipsoid mit Kreuztermen

Unser einzelner Prototypfilter kann keine rotierten Ellipsoide lernen. Eine Population entlang einer schrägen korrelierten Markerachse muss mit einer achsenparallelen Region angenähert werden. Mehrere Filter können gemeinsam komplexere Entscheidungen erzeugen, ersetzen aber nicht die fehlenden Kreuzterme eines einzelnen Filters. Nach Rücktransformation zu rohen FCS-Werten ist die Grenze wegen arcsinh im Allgemeinen kein Ellipsoid mehr.

[INTERPRETATION] Ein Prototyp ist sinnvoll, wenn eine Population durch einen begrenzten gemeinsamen Markerbereich charakterisiert ist. Er kann Werte oberhalb und unterhalb eines Zielbereichs ausschließen. Ein linearer Filter kann dagegen gerichtete Anreicherungen, breite Aktivierungsverläufe oder „je höher NKG2C/CD57 und je niedriger ein Gegenmarker, desto stärker“ einfacher darstellen. Ein radialer Filter bestraft auch das Überschreiten seines Zentrums. Keiner dieser geometrischen Vorzüge beweist, dass das jeweilige Modell im vorhandenen Datensatz besser generalisiert.

6. GrHa-Resultate

[RESULT] Die gespeicherten Resultate belegen kein überraschend gutes Radiusmodell: RM, RT, RQ

Modell	Splits	Network-AUC: Mittel / Median	Frequency-AUC: Mittel
Baseline	100	0,80750 / 0,87500	0,85587, 98 gültige Splits
Radius + Mean	100	0,63625 / 0,62500	0,58763, 97 gültige Splits
Radius + Top1	3	0,70833 / 0,75000	0,60417
Linear-quadratisch + Top1	10	0,93750 / 1,00000	0,83750

Auf den drei gemeinsamen Splits 0–2 erzielt Radius/Mean 0,54167 und Radius/Top1 0,70833; die Baseline erreicht dort 1,0. Auf den zehn quadratischen Splits erreicht die Baseline 0,875 Network-AUC und 0,90625 Frequency-AUC. Die quadratische Variante verbessert also in diesem kleinen Lauf die Network-AUC, nicht die mittlere Frequency-AUC.

Radius/Mean ist gegenüber der Baseline auf 18 der 100 Splits besser, 13 gleich und 69 schlechter; mittlere Differenz $-0,17125$. Die quadratische Variante ist auf drei der zehn Splits besser, sechs gleich und einem schlechter; mittlere Differenz $+0,0625$. Letzteres ist ein interessantes exploratives Ergebnis, kein belastbarer Nachweis überlegener Generalisierung.

Die fehlenden Phänotypwerte entstehen bei fehlendem positiven Output-Kontrast. Für einen direkten Frequenzvergleich bleiben 95 gemeinsame gültige Splits: Baseline 0,85724 versus Radius/Mean 0,59079. Die unstandardisierte mittlere Frequency-Differenz beträgt über jeweils gültige Splits 0,00740 für die Baseline und 0,02651 für Radius/Mean. Ein größerer Gruppenmittelunterschied kann somit mit schlechterer AUC einhergehen; AUC beurteilt die Rangordnung der Donoren, nicht die Größe einzelner Frequenzausschläge.

[INTERPRETATION] Lokale kompakte Zellpopulationen und eine absolute Nullantwort könnten einem Radiusmodell grundsätzlich helfen. Im vorhandenen 100-Split-Lauf ist daraus aber kein Vorteil gegenüber der Baseline entstanden. Eine plausible Erklärung für das quadratische Ergebnis ist die zusätzliche Krümmung bei Erhalt der linearen Ausgangsfunktion und ihrer Initialisierung. Welche Markerkrümmungen dafür verantwortlich sind, ist ohne gezielte Parameter- und Ablationsanalyse nicht belegt.

7. Learnable-Pooling-Architektur

[CODE] PrototypeCellCNN, Architektur-Codezelle 5: Zentren und normalisierte Diagonalgewichte bleiben erhalten. `raw_rho` und `radii()` entfallen. `responses()` liefert

$$s_k(x) = -d_k^2(x) \leq 0.$$

Nahe Zellen erhalten höhere, also weniger negative Antworten. Es gibt keine ReLU und keine absolute Nichtzugehörigkeitsgrenze. L1, L2

Pro Filter wird ein Rohparameter $\theta_k = \text{raw_alpha}_k$ gelernt:

$$\alpha_k = \text{sigmoid}(\theta_k), \quad \theta_k^{(0)} = \log \frac{0,01}{0,99}.$$

Nach absteigender Sortierung $s_{(1)k} \geq \dots \geq s_{(N)k}$ verwendet `pooled_responses()` exakt

$$q_r = \frac{r - 0,5}{N}, \quad m_{rk} = \text{sigmoid}\left(\frac{\alpha_k - q_r}{\tau}\right), \quad \tau = 0,002,$$

$$P_k(B) = \frac{\sum_{r=1}^N m_{rk} s_{(r)k}}{\max(\sum_{r=1}^N m_{rk}, 10^{-6})}.$$

Alpha ist die Rangposition, an der das Maskengewicht ungefähr 0,5 erreicht. Es ist **kein absoluter Distanzthreshold**. Zwei Donoren mit völlig unterschiedlichen Distanzen erhalten dieselben Maskengewichte für dieselben Rangpositionen. Für endliche Argumente ist die mathematische Sigmoidmaske überall positiv; numerisch können sehr kleine Gewichte auf null runden.

Die Temperatur bestimmt die Breite des weichen Übergangs. Bei $\alpha = 0,01$ liegt der Übergang von Maskengewicht 0,9 zu 0,1 zwischen etwa 0,56 % und 1,44 % der Ränge. Er umfasst damit ungefähr 26 Zellen eines 3.000-Zellen-Bags, verglichen mit 30 Zellen beim harten Top1.

[RESULT – deterministische Maskenberechnung] Bei $N = 3000$, $\alpha = 0,01$ und $\tau = 0,002$ liegen **13,84 % der normalisierten Gewichtsmasse außerhalb der besten 1 %**. Die effektive gewichtete Zellzahl $(\sum m)^2 / \sum m^2$ beträgt etwa 37,48 statt 30. Dies belegt die unterschiedliche Aggregation, nicht deren Schädlichkeit. Auch ein festes Alpha von 1 % ist deshalb keine exakte hard-Top1-Ablation. Im Grenzfall $\tau \rightarrow 0$ erfolgt zudem die Auswahl über Rangmitten; deren Rundung muss nicht exakt der bisherigen Floor-Regel entsprechen.

Sortieren verhindert das Lernen von Alpha nicht: Die Ableitung läuft durch die kontinuierliche Maske; fast überall werden Gradienten zu den entsprechend sortierten Zellantworten zurückgeführt. Bei Gleichständen ist die Rangzuordnung nicht eindeutig. Der Code verwendet hierfür PyTorch-Autograd.

Der Output und der Donorscore bleiben wie in Abschnitt 2. Für v1 gilt

$$\mathcal{L}_{v1} = \text{CE} + 10^{-4} \|V\|_F^2.$$

Im aktuellen v2-Trainingscode gilt dagegen

$$\mathcal{L}_{v2} = \text{CE} + 10^{-4} \|V\|_F^2 + 10^{-3} \frac{1}{Kd} \sum_{k,j} (a_{kj} - 1)^2.$$

Die zusätzliche Strafe erreicht über die Normalisierung `raw_a`. Zentren, Alpha und Output-Bias werden dadurch nicht regularisiert. **Early Stopping und der Loss-Tie-Breaker verwenden weiterhin nur CE plus Output-L2**, ohne Geometriestrafe. Zusätzlich wird in v2 die reine Validierungs-CE des besten Zustands protokolliert. L2

37 Marker -> arcsinh -> Inner-Train-Scaler
-> diagonale Prototypdistanz d^2
-> Zellantwort $-d^2$
-> absteigend sortieren
-> Sigmoid-Rangmaske mit Alpha und Temperatur 0,002
-> normalisiertes gewichtetes Mittel
-> Output-Logits -> Softmax
-> Mittel aus fünf Vorhersage-Bags -> Donorscore

8. Direkter Codevergleich GrHa versus Learnable Pooling

[CODE] Die folgende Tabelle beschreibt die tatsächlichen Implementierungen. M, T, L1, L2

Eigenschaft	GrHa 06b / 06c	Learnable v1	Aktuelles v2
Zentren c	trainierbar	trainierbar	unverändert
Markergewichte a	positive, normalisierte Diagonale	identisch	identisch
Radius ρ	positive Softplus-Parameter	entfernt	entfernt
Zellantwort	$\max(0, \rho - d^2)$	$-d^2$	$-d^2$
Pooling	Mean / hard Top1	weiche Rangmaske	weiche Rangmaske
Alpha	keines; 06c festes Top1	trainierbar, Start 1 %	unverändert
Temperatur	keine	0,002	0,002
Output-L2	$10^{-4} \ V\ ^2$	identisch	identisch
Geometriestrafe	keine	keine	$10^{-3} \text{mean}(a - 1)^2$
Zentrenstart	zufällige Trainingszellen	identisch	identisch
Weiterer Startparameter	Radius: Trainingsdistanz-Quantil 1 %	Alpha: 1 %	identisch
Parameterzahl	$77K + 2$	$77K + 2$	$77K + 2$
Phänotypmessung	Anteil mit $d^2 < \rho$	gepoolte Antwort	gepoolte Antwort
Refit auf 14 Donoren	nein	nein	nein

Radius und Alpha sind jeweils ein Skalar pro Filter. Der Austausch erhöht die rohe Parameterzahl gegenüber GrHa deshalb **nicht**. Gegenüber der linearen Baseline ist das Prototypmodell aber größer:

Filterzahl K	Linear: $40K + 2$	Prototyp / linear-quadratisch: $77K + 2$
3	122	233
4	162	310
5	202	387

Die Zahlen wurden an den tatsächlichen PyTorch-Klassen geprüft. Die Normalisierung von a beschränkt dessen wirksame Form auf $d - 1$ Freiheitsgrade je Filter; die rohe Parameterzählung allein ist daher kein vollständiges Komplexitätsmaß.

[INTERPRETATION] Schon von 06c zu v1 ändern sich Radius, ReLU, Response, hartes/weiches Pooling und die Phänotypdefinition gleichzeitig. Von 06b kommt der Wechsel vom Gesamtmittel hinzu. Ein Performanceunterschied zwischen diesen Branches isoliert Alpha nicht. Der Vergleich zu v2 enthält darüber hinaus eine zusätzliche Regularisierungsänderung.

9. Warum die Performance sinkt

Was tatsächlich gemessen wurde

[RESULT] Im 100-Split-Vergleich v1 gegen Baseline sind deine Zahlen bestätigt: R100, SEL100

Kennzahl	Baseline	Learnable v1
Test-AUC, Mittel	0,80750	0,71875
Test-AUC, Median	0,87500	0,75000
Validierungs-AUC ausgewählter Modelle, Mittel	0,96833	0,91750
Validierungs-AUC ausgewählter Modelle, Median	1,00000	1,00000
Validierungs-Accuracy ausgewählter Modelle, Mittel	0,93800	0,88400
Kandidatenbewertungen	900	900
Epochen aller Kandidaten, Median	8	7
Epochen ausgewählter Kandidaten, Median	18,5	26
Kandidaten mit 100 gelaufenen Epochen	19	9

Die mittlere gepaarte Differenz v1 minus Baseline ist **−0,08875**, ihr Median **0**. v1 ist auf **27 Splits besser, 24 gleich und 49 schlechter**. Die Differenz der AUC-Mediane, **−0,125**, ist nicht der Median der gepaarten Differenzen.

Über alle 411 Filter der 100 ausgewählten v1-Modelle liegt Alpha bei:

Alpha-Statistik	Wert
Minimum / Maximum	0,1346 % / 16,4682 %
Median	1,6413 %
Mittlere 50 % der Werte	1,0130 % bis 2,7779 %
Median beim stärksten positiven Filter	1,9063 %

Das Modell verändert den Poolingbereich tatsächlich. Die Tabelle gewichtet Filter gleich; Modelle mit fünf Filtern tragen mehr Zeilen bei als Modelle mit drei. Es sind weder 411 unabhängige Experimente noch gemessene Zellfrequenzen. A100

Für v2 sind auf Splits 0–49 Baseline/v1/v2 **0,8025 / 0,7200 / 0,7225**. Gegenüber v1: drei Splits besser, 45 gleich, zwei schlechter. Die 450 Kandidatenbewertungen und 300 Testvorhersagen passen zu 50 vollständigen Splits. R50

Bei v2 beträgt der Alpha-Median über 211 Filter 1,7108 %, die Spannweite 0,4437–22,0185 %; beim stärksten positiven Filter liegt der Median bei 1,9240 %. Die ausgewählte Validierungs-AUC beträgt im Mittel 0,91667. Kandidaten laufen im Median sieben Epochen, ausgewählte Kandidaten 30,5; fünf von 450 Kandidaten erreichen 100 Epochen. Auch diese Angaben stammen aus den lokalen v2-Auswahl- und Alpha-Dateien, nicht aus der alten v1-Tabelle.

Welche Ursachen die Befunde stützen

[INTERPRETATION – gestützt] Kleine Validierungsgruppen und Auswahloptimismus sind ein wesentliches Problem der Unsicherheit. Aus neun Modellen wird das auf vier oder fünf Donoren erfolgreichste ausgewählt. Bei v1 liegt die ausgewählte Validierungs-AUC im Mittel um 0,19875 über der Test-AUC, bei der Baseline um 0,16083. Das sind unterschiedlich ausgewählte Datengruppen, kein sauber isoliertes Overfitting-Maß; die Lücke belegt aber, dass die gewinnende Validierungsleistung kein belastbarer Erwartungswert für den Test ist. Schon die v1-Validierung liegt

unter der Baseline. „V1 trainiert perfekt und scheitert ausschließlich im Test“ wäre daher nicht durch die vorliegenden Zahlen gedeckt.

Die Auswahl über verschiedene Folds vergleicht zudem unterschiedliche kleine Validierungsmengen. Der vier Donoren große Fold 2 wird bei der Baseline in 52 und bei v1 in 47 von 100 Splits ausgewählt. Das belegt keine Ursache allein, zeigt aber, dass Filterzahl und Trainingssubset gemeinsam selektiert werden. Die verglichenen finalen Netze müssen dadurch nicht dieselben Trainingsdonoren verwendet haben.

[INTERPRETATION – plausible Hypothese] Geometrie und Initialisierung könnten den Optimierungsverlauf erschweren. Die Prototypmodelle müssen Zentrum und Markerpräzision einer nützlichen Population finden. Bei einer zufällig gezogenen Initialzelle und einem Anteil p geeigneter Zellen in der balancierten Referenz beträgt die ungefähre Wahrscheinlichkeit, dass mindestens eines von K Zentren diese Population trifft, $1 - (1 - p)^K$. Für $p = 1\%$ und $K = 3$ bis 5 sind das nur etwa 3–5 %. Das ist eine hypothetische Illustration; der tatsächliche Anteil der gesuchten Population ist nicht bekannt. Zentren können anschließend wandern, sodass ein verfehlter Start nicht automatisch ein verlorener Filter ist.

Die lineare Baseline braucht keinen zufälligen Punkt innerhalb einer seltenen Population. Das quadratische Modell startet sogar exakt bei der linearen Funktion. Dies sind konkrete Unterschiede, aber noch kein Nachweis, dass die Initialisierung den beobachteten AUC-Abstand verursacht.

[INTERPRETATION – plausible Hypothese] Zusätzliche Flexibilität kann bei wenigen Donoren überanpassen. Das Prototypmodell hat gegenüber der Baseline etwa die doppelte Parameterzahl und lernt auch den Aggregationsbereich. Gegenüber GrHa ersetzt Alpha jedoch Radius; dort gibt es keinen Anstieg der Parameterzahl. Die Alpha-Verteilung belegt Anpassung, nicht deren Überanpassung.

[INTERPRETATION – plausible Hypothese] Die weiche Maske könnte den Rare-Cell-Fokus abschwächen. Ihre Gewichtsverteilung ist nachweislich breiter als hard Top1, und viele gelernte Alphas überschreiten 1 %. Ob diese Breite störenden Hintergrund aufnimmt oder nützliche Robustheit schafft, beantwortet erst die kontrollierte Ablation.

[INTERPRETATION – begrenzte Evidenz] Fehlende Geometrieschrumpfung war keine hinreichende Erklärung. Auf den ersten 50 Splits sinkt der mittlere Wert $\text{mean}(a - 1)^2$ der ausgewählten Modelle von etwa 1,6414 auf 1,4355. Die AUC steigt dennoch nur um 0,0025. Die konkrete Strafe wirkt auf die Parameter, liefert bislang aber keinen überzeugenden Leistungsgewinn. Eine stärkere Strafe wäre ein neues Experiment, keine daraus folgende Lösung.

Early Stopping allein erklärt den Abstand nicht. Die ausgewählten v1-Modelle laufen im Median länger als die Baseline; nur neun von 900 Kandidaten erreichen das Epochenlimit. Der Code erzeugt Lernhistorien, speichert diese aber nicht in den untersuchten Checkpoints oder Auswahl-CSV-Dateien. Vollständige Verlustkurven und der explizite beste Epochindex sind dort nicht nachweisbar. Bei einem Stopp vor 100 Epochen lässt sich der beste Epochindex als `epochs_run - 5` rekonstruieren; bei Erreichen des Limits gilt das nicht allgemein.

Schließlich ist die Richtung des GrHa-Vergleichs zu beachten: **v1 ist gegenüber Radius/Mean über dieselben 100 Splits im Mittel um 0,0825 besser**, mit 51 besseren, 24 gleichen und 25 schlechteren Splits. Die Daten rechtfertigen deshalb keine Erzählung, wonach allein die Entfernung des Radius die Performance verschlechtert habe.

10. Phenotype-Scoring und Clustering

Was im Learnable-Pooling-Branch „Phenotype“ heißt

[CODE] Wieder wird $k^* = \arg \max_{\Delta_k > 0} \Delta_k$ gewählt. Für jeden Testdonor wird auf derselben festen bis zu 20.000 Zellen großen Evaluationsstichprobe wie in 06a die einzelne Antwort

$$g_i = P_{k^*}(E_i)$$

berechnet. Sie wird **nicht** über die fünf Network-Prediction-Bags gemittelt. `pooled_response_auc` ist $AUC(g)$; `pooled_response_effect` ist $\bar{g}_+ - \bar{g}_-$. Ohne positiven Kontrast bleiben diese Werte fehlend. Eine konstante Antwort ergibt AUC 0,5, solange beide Klassen vorhanden sind. L1, L2

[RESULT] v1 erreicht eine mittlere `pooled_response_auc` von 0,7450 und einen mittleren Antwortunterschied von 0,95937 auf 100 gültigen Splits. Für v2 sind es 0,7525 und 0,88745 auf 50 Splits. Der Unterschied hat Einheiten der gewichteten quadrierten Standarddistanz, keine Prozentpunkte. Da Filter und Metriken zwischen Splits wechseln, ist auch sein Mittel keine einheitliche biologische Effektgröße. R100, R50

[INTERPRETATION] Das ist ein zulässiger Score für die CMV-Assoziation eines gelernten Filtermerkmals. Es ist keine biologische Validierung eines Zelltyps und keine gemessene Populationshäufigkeit. Ein positiver Output-Kontrast bedeutet bei negativen Antworten: Eine weniger negative Antwort erhöht den CMV+-Logitkontrast. Er bedeutet nicht, dass alle ausgewählten Zellen CMV-spezifisch sind.

Was sich ohne Zelltyp-Ground-Truth evaluieren lässt

Die verfügbaren CMV-Labels gehören zu Donoren. **Echte Zelltyp-Labels für sämtliche Einzelzellen sind im Repository nicht nachweisbar.** `task3_annotations.csv` enthält nachträgliche markerbasierte Beschreibungen wie „NK-kompatibel“ oder „CD8-T-kompatibel“, keine unabhängige Ground Truth. `gated_NK` liefert eine manuelle grobe Kompartimentauswahl; daraus folgt ohne verifizierte Ereigniszuordnung keine vollständige Zelltypannotation für `gated_alive`. D3

Auswertungsfrage	Geeignete Größe	Was sie nicht beweist
Sagt das Modell CMV voraus?	Donor-Test-AUC	Korrekte Zelltypen
Ist eine definierte Population CMV-assoziiert?	Frequency-AUC und Gruppenunterschied	Biologische Reinheit
Wird dieselbe Population wiedergefunden?	Überlappung, Markerprofil- und Zentroidstabilität	Unabhängige Replikation
Ist das Profil biologisch plausibel?	Markerkoexpression und Expertenabgleich	Bestätigte Zellidentität
Zerfällt das Subset in mehrere Gruppen?	Clustering und dessen Stabilität	Dass jede Gruppe CMV-relevant ist

Für die im Paper diskutierte Population sind insbesondere CD56, CD3, NKG2C und CD57 relevant. Alle vier werden gemessen. Ein sinnvoller Befund wäre ein nachvollziehbares gemeinsames Profil; getrennte hohe Marker-Mittelwerte beweisen keine Koexpression derselben Zellen. Die lokalen markerbasierten Annotationen dürfen zur Plausibilisierung, aber nicht als unabhängige Wahrheit für ARI/NMI oder eine „Zelltyp-Accuracy“ verwendet werden. P, D3

Für Masken zweier Modelle auf **denselben Originalzellen** ist beispielsweise

$$J(C^{(a)}, C^{(b)}) = \frac{|C^{(a)} \cap C^{(b)}|}{|C^{(a)} \cup C^{(b)}|}$$

eine mögliche Stabilitätsgröße. Jede Maske muss mit dem zu ihrem Modell gehörenden Scaler berechnet werden. Leere Masken und unterschiedliche Populationsgrößen sind separat auszuweisen. Eine gemeinsame Karte oder Referenz aus allen Donoren ist für eine nachträgliche Beschreibung möglich, liefert aber keine unabhängige prospektive Stabilitätsprüfung. Wiederkehr über überlappende Outer-Splits ist interne Stabilität, keine neue biologische Replikation.

[RECOMMENDATION] Für ein zusätzliches Clustering zunächst einen Filter und seine Schwelle ausschließlich anhand der Trainingsdaten festlegen. Aus dessen ausgewählten Zellen spenderbalanciert eine Trainingsreferenz ziehen, darin clustern und das Zuordnungsverfahren einfrieren. K-Means wäre für einen ersten kontrollierten Versuch praktisch, weil Testzellen unmittelbar eingefrorenen Zentren zugeordnet werden können. Bei Ward oder Leiden muss die Zuordnung neuer Zellen gesondert definiert werden; erneutes Clustering aller Testzellen wäre ein anderes, transduktives Verfahren.

Für Cluster g sollte die Häufigkeit

$$f_{ig} = \frac{\#\{\text{Donorzellen im ausgewählten Subset und Cluster } g\}}{\#\{\text{evaluierte Zellen des Donors } i\}}$$

auf alle evaluierten Donorzellen bezogen werden. Eine Häufigkeit relativ nur zum ausgewählten Subset beantwortet eine andere Frage. CMV-Enrichment wird spenderweise anhand dieser Häufigkeiten beurteilt, nicht durch einen Test, der Millionen Zellen als unabhängige Beobachtungen behandelt. Ein nachträglich auf Testlabels ausgewählter „bester Cluster“ wäre Ergebnisoptimierung am Testset.

Ohne Zelltyp-Ground-Truth gibt es daher keinen einzelnen objektiven Sieger-Score für die „beste Phänotyppopulation“. Eine überzeugende Population kombiniert auf zurückgehaltenen Donoren reproduzierbare Assoziation, stabile Zellselektion und ein plausibles gemeinsames Markerprofil. AUC allein reicht nicht; ein hoher Silhouette-Wert ebenfalls nicht.

Übertragung der Baseline-Frequenzlogik

Eine **absolute, auf Trainingsdaten bestimmte und danach eingefrorene Schwelle** lässt sich auch ohne trainierbaren Radius verwenden:

$$C_{ik} = \{x : d_k^2(x) < r_k\}, \quad f_{ik} = |C_{ik}|/N_i.$$

Ein einfacher vorab festgelegter Ansatz ist $r_k = Q_{0,01}$ der Distanzen in einer spenderbalancierten Trainingsreferenz. Danach können Frequenz-AUC und Frequenzeffekt auf Testdonoren berechnet werden. Dies ändert nicht die Netzwerkvorhersage; es ergänzt eine interpretative Populationsdefinition. Sie ist allerdings nicht identisch mit der Halbmaximum-Regel der Baseline.

Die Halbmaximum-Regel darf nicht wortwörtlich auf negative Antworten übertragen werden. Ist $\max s = -a < 0$, dann liegt $\frac{1}{2} \max s = -a/2$ oberhalb der größten Trainingsantwort; die so definierte Trainingspopulation wäre leer. Bei negativen Distanzscores ist ein Trainingsquantil beziehungsweise eine positive Distanzgrenze sinnvoller.

Leakage entstünde, wenn die Grenze oder der Filter anhand der Outer-Test-Labels optimiert würde oder die Referenz lernend aus Testdonoren gebildet würde. Nimmt man stattdessen bei jedem Testdonor stets genau dessen beste $\alpha\%$, ist die resultierende Häufigkeit ungefähr α – abgesehen von Rundung und Gleichständen. Sie misst dann die vorgegebene Auswahlquote, nicht das Vorkommen einer Population.

[CODE – lokale Ausführbarkeit] Im lokalen Stand 68452ce wurde SavedCellCNN aus `src/task_5_interpretation.py` entfernt. Die vorhandenen Bonusnotebooks importieren die Klasse weiterhin; dieser Import wurde geprüft und schlägt fehl. Im untersuchten GitHub-Stand 4477d6b und im Learnable-Pooling-Worktree ist die Klasse vorhanden. Historische Resultate wurden deshalb mit den zugehörigen Quellständen rekonstruiert. Die gespeicherten Outputs beweisen nicht, dass jedes aktuelle lokale Notebook mit frischem Kernel durchläuft. IL, I, F

11. Datenmenge, CV und Refit

[CODE] Pro Outer-Split sind 14 Donoren für Entwicklung verfügbar, aber nur neun oder zehn trainieren das final getestete Netz. Alle drei Inner-Folds zusammen haben zwar jeden Outer-Train-

Donor zum Training verwendet – jedoch in unterschiedlichen Modellen. Das ausgewählte Einzelmodell hat dadurch keine gemeinsamen Parameter aus einem Fit auf allen 14 Donoren. B, L2

[RECOMMENDATION] Ein Refit ist statistisch sauber und als Erweiterung sinnvoll zu testen: Inner-CV bestimmt Konfiguration und Trainingsbudget; danach werden Scaler und Netz neu auf allen 14 Outer-Train-Donoren gefittet, und erst anschließend erfolgt die einmalige Outer-Test-Auswertung. Das erhöht die Zahl der Donoren im Gradienten-Training um 40 % gegenüber zehn beziehungsweise rund 56 % gegenüber neun. Es erhöht nicht die Gesamtzahl unabhängiger Donoren des Datensatzes.

Für Early Stopping darf dabei kein Outer-Test-Donor verwendet werden. Eine konkrete Lösung ist, für die ausgewählte Filterzahl die besten Epochindizes aus den drei Inner-Folds zu speichern und deren gerundeten Median als feste Refit-Epochenzahl zu verwenden. Danach gibt es im Refit keinen validierungsbasierten Stopp. Ein erneuter interner Holdout wäre ebenfalls zulässig, würde aber wieder einen Teil der 14 Donoren vom abschließenden Fit ausschließen. Ein Refit garantiert keinen AUC-Gewinn: Optimierung und geeignetes Trainingsbudget können sich mit der größeren Stichprobe ändern.

Die historische Auswertung sollte dabei erhalten bleiben. Das Paper testet für den NK-Benchmark ebenfalls ein ausgewähltes Inner-Fold-Netz. Der neue Refit ist eine ausdrücklich gekennzeichnete Protokolländerung, nicht die rückwirkende „Korrektur“ eines vermeintlich unzulässigen Tests. P

Maßnahme	Was zusätzlich nutzbar wird	Neue unabhängige Donoren?
Mehr Zellen je Donor	Präzisere Populationsdarstellung; bessere Erfassung seltener Zellen	Nein
Mehr beziehungsweise neu gezogene Bags	Mehr Monte-Carlo-Trainingsansichten	Nein
Refit auf allen Outer-Train-Donoren	Bisherige Validierungsdonoren im Gradienten-Training	Mehr je finalem Fit, aber keine neuen Personen
Mehr Seeds oder Ensembles	Möglicherweise geringere Optimierungsvarianz	Nein
Externes geeignetes Pretraining	Zusätzliche Struktur aus unabhängigen Daten	Nur bei tatsächlich externen Donoren
Pretraining auf allen 20 Projekt-Donoren	Testinformationen im Modellaufbau	Kein sauberer induktiver Outer-Test
Tatsächlich neue Donoren	Zusätzliche biologische Beobachtungen	Ja

Bei zufälligem Sampling aus einem Donor mit Populationsanteil p enthält ein Bag der Größe N mit Wahrscheinlichkeit $1 - (1 - p)^N$ mindestens eine solche Zelle. Für hypothetische $p = 0,01\%$ und $N = 3000$ sind das etwa 25,9 %. Mehr Zellen können die Erfassung einer vorhandenen seltenen Population also verbessern. Sie ersetzen keine neuen Personen und keine zusätzliche Variation zwischen Donoren.

Die 100 Outer-Splits dürfen nicht zu einem zusätzlichen Trainingsdatensatz zusammengeworfen werden. Derselbe Donor ist je nach Split trainierend oder testend. Für das Modell eines bestimmten Splits bleiben dessen sechs Testdonoren vollständig ausgeschlossen. Würde man dafür Trainingsdaten anderer Splits vereinigen, kämen die eigenen Testdonoren wieder hinein. Das wäre Leakage, auch bei unüberwachtem Pretraining.

Nach Abschluss aller Methodenauswahl kann ein separates finales Nutzungsmodell auf allen 20 Donoren trainiert werden. Seine Leistung auf diesen 20 Donoren ist aber keine neue Testleistung. Zur Bewertung dieses endgültigen Fits wären unabhängige Daten erforderlich.

Auch die statistische Auflösung ist begrenzt: Vier negative und zwei positive Testdonoren ergeben nur acht positive-negative Paare. Ohne Gleichstände verändert sich die Split-AUC in Schritten von $1/8$, mit Gleichständen sind halbe Schritte möglich. 600 Testvorhersagen über 100 Wiederholungen

sind deshalb keine 600 unabhängigen Fälle. Ein gewöhnlicher Standardfehler über 100 vermeintlich unabhängige Split-AUCs wäre irreführend. Die hier genannten Vergleiche sind deskriptiv.

12. ReLU-Radius versus Alpha

[CODE / MATHEMATIK] Im Radiusmodell ist

$$\rho_k = \text{softplus}(\text{raw_rho}_k) + 10^{-6}, \quad h_k(x) = \max(0, \rho_k - d_k^2(x)).$$

ρ_k ist eine Schwelle auf der **quadrierten gewichteten Distanz**, nicht unmittelbar ein euklidischer Radius. Es gilt exakt $h_k(x) > 0 \iff d_k^2(x) < \rho_k$. Die Response ist kontinuierlich; erst die Bedingung $h > 0$ erzeugt eine harte Membership-Grenze. Das ist kein exklusives Cluster-Assignment: Eine Zelle darf zu mehreren Filtern oder zu keinem gehören. M

Hat Donor A Zellen mit $d^2 = 0,1$ und Donor B nur Zellen mit $d^2 \geq 10$, liefert ein Radius $\rho = 0,5$ für passende A-Zellen Antwort 0,4 und für sämtliche B-Zellen null. Auch Top-Pooling bleibt bei B null. Bei $s = -d^2$ liefern die besten Zellen hingegen etwa $-0,1$ beziehungsweise höchstens -10 . Das Pooling berücksichtigt in beiden Donoren höchststrangige Zellen, auch wenn alle schlecht passen.

Die absolute Distanzinformation verschwindet bei $-d^2$ trotzdem nicht. Das Netzwerk kann die stark unterschiedlichen negativen Werte unterscheiden. Was fehlt, ist die explizite Nullantwort als Mitgliedschaftsaussage. Es wäre mathematisch falsch zu behaupten, rank-basiertes Pooling mache gute und schlechte Donoren zwangsläufig ununterscheidbar.

Beide Parameter erfüllen verschiedene Funktionen:

$$d_k^2 \longrightarrow \text{ReLU}(\rho_k - d_k^2) \longrightarrow \text{SoftTopMean}_{\alpha_k}.$$

ρ bestimmt grundsätzliche Zugehörigkeit; α bestimmt die Breite der Aggregation starker Antworten. Bei Mean-Pooling gilt exakt „Häufigkeit der positiven Antworten $\times$ mittlere Antwort innerhalb der Population“. Top-Pooling verändert diese Gewichtung und kann eine sehr kleine Population hervorheben.

Beide lernbar zu machen kostet gegenüber den aktuellen Prototypmodellen nochmals K Parameter, insgesamt $78K + 2$. Entscheidend ist weniger diese kleine Zahl als ihre gekoppelte Wirkung: Radius verändert Häufigkeit und Antwortstärke, Alpha die Aggregation. Bei 20 Donoren würde ich beide nur mit einer festen-Alpha-Kontrolle gemeinsam testen, nicht ungeprüft als Verbesserung übernehmen. ReLU kann außerdem Filter ohne aktive Trainingszellen vorübergehend ohne Gradienten für Zentrum und Radius lassen; eine harte Nullgrenze ist daher nicht ausschließlich vorteilhaft.

13. Regularisierung

[CODE] Die aktuelle Situation lautet: B, L1, L2

Parameter	Baseline	Learnable v1	Learnable v2
Output-Gewichte V	L2	L2	L2
Output-Bias β	keine Strafe	keine Strafe	keine Strafe
Lineare Filter W	L2	nicht vorhanden	nicht vorhanden
Zentren c	nicht vorhanden	keine Strafe	keine Strafe
<code>raw_a</code>	nicht vorhanden	keine Strafe	indirekt über $\text{mean}(a - 1)^2$
<code>raw_alpha</code>	nicht vorhanden	keine Strafe	keine Strafe
Radius	nicht vorhanden	nicht vorhanden	nicht vorhanden

Output-L2 begrenzt den Klassifikationskopf, aber nicht unmittelbar die Filtergeometrie. Das ist ein zulässiges Design, jedoch kein vollständiges Analogon zum L2 der linearen Baseline. Dass Gradienten der Cross Entropy zu Zentren und Alpha gelangen, bedeutet nicht, dass diese Parameter regularisiert sind.

[RECOMMENDATION] Priorität 1: die vorhandene normierte Markergewicht-Shrinkage sachlich bewerten, nicht weitere Strafen stapeln. Ihr Nullpunkt $a_j = 1$ ist geometrisch verständlich und berücksichtigt die Normalisierung. Der bestehende 50-Split-Test zeigt wenig Einfluss auf die AUC. Für kausale v1-Ablationen sollte der Koeffizient zunächst null bleiben; v2 wird als gesondertes Experiment behandelt.

Priorität 2: ein fester Alpha-Wert als einfachere Kontrolle vor einem Alpha-Prior. Falls die Ablation einen Nutzen enger Poolingbereiche zeigt, wäre etwa eine Strafe $(\theta_k - \text{logit}(0,01))^2$ ein gezielter Prior. Sie führt aber einen neuen Koeffizienten ein. Direktes L2 auf `raw_alpha` zieht den Parameter dagegen zu null und damit **Alpha zu 50 %**, nicht zu 1 %.

Zentren-L2 würde ich zunächst nicht hinzufügen. $\|c\|^2$ zieht Zentren zum Nullpunkt der standardisierten Mischung. Dieser muss keinem biologisch sinnvollen Zelltyp entsprechen. Insbesondere weit vom globalen Mittel liegende seltene Populationen könnten dadurch benachteiligt werden. Eine Bindung an Initialzentren würde wiederum deren zufällige Auswahl konservieren.

Weitere Ideen haben derzeit niedrigere Priorität:

- **Strafe auf $\log a$:** kann extreme Achsenverhältnisse begrenzen, ist aber eine alternative Geometriestrafe mit anderem Verhalten bei kleinen Gewichten; nicht zusätzlich ohne gezielte Hypothese einsetzen.
- **Entropie-/Sparsity-Strafe:** verlangt eine klare Zielrichtung. Sparse Markergewichte können die Auswahl auf wenige Marker konzentrieren und andere Achsen sehr breit machen; das ist nicht automatisch biologisch besser.
- **Radiusstrafe:** erst relevant, wenn Radius wieder vorhanden ist. Kleine Radien können seltene Populationen fokussieren, aber auch leere oder kaum trainierbare Filter erzeugen.
- **Pauschaler Weight Decay:** ist wegen der Rohparametrisierungen kein neutraler Ersatz. Bei `raw_rho` = 0 wäre beispielsweise $\rho \approx 0,693$, nicht null. Bei Adam ist zusätzlicher entkoppelter Weight Decay außerdem nicht einfach dieselbe Operation wie die vorhandene explizite L2-Loss.

Mit so wenigen Donoren ist ein weiterer Regularisierungskoeffizient auch eine weitere Modellselektionsentscheidung. Eine große Suche über diese Ideen wäre methodisch weniger überzeugend als die gezielten Kontrollen.

14. Konkrete Empfehlungen

Ich wähle Option B: kleine Änderungen. Option A würde eine biologisch missverständliche Interpretationsmetrik und ungeklärte Architekturkonfundierung belassen. Für Option C, etwa volle Mahalanobis-Matrizen oder tiefere Netze, gibt es derzeit keine ausreichende Evidenz.

Die drei Prioritäten sind:

1. **Die Poolingänderung isolieren.** Zuerst aktuelles $-d^2$ mit derselben weichen Maske und festem Alpha 1 % gegen lernbares Alpha vergleichen. Danach Radius/ReLU bei konstant gehaltener Poolingart untersuchen. Das beantwortet unmittelbar, ob der zusätzliche Alpha-Freiheitsgrad oder ein anderer gleichzeitiger Umbau verantwortlich sein könnte.
2. **Refit auf alle 14 Outer-Train-Donoren separat testen.** Dabei Konfiguration, Seedregel und aus Inner-CV bestimmtes Trainingsbudget fixieren. Die historische papernahe Auswertung als Referenz behalten. Das adressiert den größten konkret vorhandenen Unterschied zwischen verfügbaren und tatsächlich gemeinsam trainierenden Donoren.
3. **Interpretation als echte Populationsmessung ergänzen.** Eine vorab definierte absolute Trainings-Distanzgrenze einfrieren, auf Testdonoren Frequenzen berechnen und Markerprofil/Stabilität berichten. Die bisherige pooled-response-AUC weiterhin als Filterassoziation bezeichnen.

Deine Varianten bewerte ich entsprechend:

Vorschlag	Urteil
A: gleiches Soft-Pooling, Alpha fest 1 %	Höchster unmittelbarer Erkenntnisgewinn für den Alpha-Effekt
B: ReLU-Radius behalten, Alpha lernen	Sinnvoll, aber zusätzlich gegen Radius mit festem weichem Alpha prüfen
C: Refit auf alle Outer-Train-Donoren	Hohe Priorität für Datennutzung; eigener Protokollvergleich
D: stärkere Regularisierung	Gegenwärtige Strafe bereits getestet; weitere Änderungen nachrangig
E: bessere Prototypinitialisierung	Plausible spätere Hypothese; bislang kein isolierter Wirkungsnachweis

Die quadratische Variante verdient wegen des kleinen positiven Ergebnisses einen späteren breiteren Vergleich. Sie ersetzt jedoch nicht die Alpha-Ablation und sollte auf Grundlage von zehn Splits noch nicht zum neuen Hauptmodell erklärt werden. Die lokale **SavedCellCNN**-Importinkonsistenz muss vor einem erneuten Notebooklauf behoben oder durch Verwendung des passenden historischen Quellstands vermieden werden; diese Analyse verändert dafür keinen Modellcode.

15. Kleine kontrollierte Ablationsstudie

[RECOMMENDATION] Für die Architekturfrage genügen die folgenden Varianten. Drei davon sind neue Kontrollen; die anderen sind vorhandene Architekturreferenzen. Ein vollständiger Benchmark wurde für diese Analyse nicht gestartet.

Kürzel	Zellantwort	Pooling	Zweck
L	linear + ReLU	hard Top1	bestehende Baseline
M	Prototyp + ReLU-Radius	Mean aller Zellen	GrHa 06b
R	Prototyp + ReLU-Radius	hard Top1	GrHa 06c
RF	Prototyp + ReLU-Radius	Soft, Alpha fest 1 %	Weichheit isolieren
RL	Prototyp + ReLU-Radius	Soft, Alpha lernbar	Alpha bei vorhandenem Radius
NF	$-d^2$	Soft, Alpha fest 1 %	Alpha im aktuellen Modell isolieren
NL	$-d^2$	Soft, Alpha lernbar	Learnable v1

Die zentralen Vergleiche sind **NF**↔**NL** für Alpha ohne Radius, **RF**↔**RL** für Alpha mit Radius, **R**↔**RF** für hartes versus weiches Pooling und **NF**↔**RF** für Radius/ReLU bei festem weichem Pooling. **M**↔**R** prüft Gesamtmittel versus seltenheitsbetonte Aggregation. L dient als Leistungsreferenz; sein Vergleich mit Prototypen ist wegen anderer Parametrisierung und nativer Eingangs-L2 kein vollständig isolierter einzelner Geometrieeffekt.

Gemeinsames Protokoll

- Originale Outer-Splits, Labels, **gated_alive**, 37 Marker, arcsinh und deterministische Bagziehungen beibehalten. Gleiche Kandidaten-Trainingsdonoren und dieselben Zellen für alle Varianten verwenden; Scalerstichproben innerhalb eines Splits/Folds zwischen Varianten teilen.
- Die vorhandenen Filterzahlen 3, 4 und 5 sowie drei Inner-Folds verwenden. Lernrate 0,01, Batchgröße 128, maximal 100 Epochen, Patience 5, Trainings-Bags 200×3000 je Donor und fünf Vorhersage-Bags mit 20.000 Zellen beibehalten. Beste Epochindizes ausdrücklich speichern.
- Für die Architekturstudie Filterzahl nach **mittlerer Donor-Validierungs-Accuracy über die drei Folds**, danach mittlerer Validierungs-AUC, danach kleinerer Filterzahl wählen. Anschließend jedes Modell auf **allen 14 Outer-Train-Donoren** refitten; feste Epochenzahl ist der auf die nächste ganze Zahl aufgerundete Median der drei besten Epochindizes für diese Filterzahl. So verwenden die final getesteten Architekturen dieselben Trainingsdonoren.
- Refit-Scalerstichprobe und Bagziehungen aus festen, architekturunabhängigen Teilseeds erzeugen. Beispielsweise **split_seed +800000** für die Scalerstichprobe, **+810000** für Trainings-Bags und **+820000** für die Modellinitialisierung; die bisherigen Test-Seeds beibehalten. Initialzentren und initiale Output-Gewichte bei gleichem Split/Fold/ K zwischen Prototypvarianten exakt teilen.
- Alle Prototypvarianten zunächst mit Output-L2 10^{-4} und Geometrikoeffizient **0** vergleichen, entsprechend der historischen v1-Frage. Die Radiusinitialisierung bleibt das Trainingsdistanz-Quantil 1 %. Alpha startet immer bei 1 %, Temperatur bleibt 0,002. „Festes Alpha“ bedeutet eingefrorener Rohparameter bei derselben numerischen Maske.

Dieses vereinheitlichte Refit-Protokoll ist eine neue, für alle Architekturen gleiche Auswertung. Seine Ergebnisse dürfen nicht als direkte Wiederholung der historischen Tabellen bezeichnet werden. Es verhindert, dass Architektur A zufällig auf anderen neun Donoren getestet wird als Architektur B.

Refit-Effekt gesondert isolieren

Für L und NL zusätzlich je Outer-Split die ausgewählte Filterzahl und das Trainingsbudget fixieren. Zwei frische Fits mit derselben Seedregel vergleichen: einer auf den tatsächlichen Trainingsdonoren des historisch ausgewählten Inner-Folds, einer auf allen 14 Outer-Train-Donoren. Es findet keine erneute Hyperparameterwahl zwischen diesen beiden Fits statt. Damit untersucht der Vergleich die Erweiterung des Trainingssets; die historischen ausgewählten Checkpoints bleiben eine zusätzliche, getrennt gekennzeichnete Referenz.

Umfang und Auswertung

Zuerst deterministische Funktionsprüfungen und ein kleiner technischer Lauf, danach ein vorab festgelegter explorativer Pilot auf Splits 0–9. Keine Variante anhand dieses Piloten fortlaufend verändern und anschließend denselben Test als bestätigenden Nachweis ausgeben. Ein abschließender 100-Split-Lauf wäre ein ausdrücklich separater Benchmark mit zuvor eingefrorenem Protokoll.

Primär werden gepaarte Donor-Test-AUCs, Mittel/Median der Differenzen und besser/gleich/schlechter berichtet. Ergänzend: Alpha, Zahl tatsächlich ausgewählter Zellen unter eingefrorener absoluter Grenze, Frequency-AUC, Effekt und Markerprofilstabilität. Dieselbe Auswahlregel und dieselben Evaluationszellen gelten für alle Varianten; fehlende Populationen bleiben sichtbar.

Da die vorhandenen Outer-Test-Ergebnisse bereits die Methodenentwicklung beeinflusst haben, bleibt auch eine weitere Auswertung derselben 20 Donoren explorativ. Mehr wiederholte Splits beseitigen diese Rückkopplung nicht. Eine unabhängige Bestätigung erfordert neue, bei der Entwicklung unberührte Donoren.

Konkrete nächste Schritte

1. Code-/Ergebnisversionen wie unten festhalten und vor neuen Läufen die lokale Bonus-Importinkonsistenz auflösen.
2. Zuerst die feste-Alpha-Kontrolle NF und die Radius-Kontrollen RF/RL mit identischen übrigen Einstellungen vorbereiten.
3. Refit und absolute Frequenzmessung gemäß dem getrennten Protokoll ergänzen; erst nach kleinen erfolgreichen Checks einen neuen Benchmark beauftragen.

Appendix: Code- und Quellenreferenzen

Untersuchte Versionen und Funktionen

- **B Baseline:** GitHub-GrHa, Commit 4477d6b3ebd9e4305423a48af9940b9c7d23a866, notebooks/04c_cellcnn.ipynb: CellCNN, materialize_multicell_inputs, fit_balanced_scaler, regularized_loss, train_candidate, predict_donor, train_outer_split. Das Notebook ist im untersuchten Learnable-Branch identisch.
- **S Splits:** derselbe Commit, notebooks/04a_data_qc_and_splits.ipynb, Code zur Outer-Ziehung und StratifiedKFold.
- **F Baseline-Frequenzen:** derselbe Commit, notebooks/06a_cellcnn_baseline_bonus.ipynb, strongest_positive_filter, summarize_split und Auswertungsschleife in Codezelle 6.
- **I GitHub-Interpretation:** derselbe Commit, src/task5_interpretation.py, SavedCellCNN, cellcnn_masks, run_python_methods, aggregate_frequencies.
- **IL Neuere lokale Interpretation:** Commit 68452ce3470ab25eb8a9bb3dd8dbcc71abf6d3ce, src/task5_interpretation.py, training_reference, halfmax_centroids, cellcnn_centroids, group_centroids, representative_cells. Dieser lokale Stand liegt vier Commits vor dem abgefragten GitHub-Branch.
- **M Radius/Mean und T Radius/Top1:** GitHub-GrHa bei 4477d6b, notebooks/06b_cellcnn_mahalanobis_relu_threshold.ipynb und notebooks/06c_cellcnn_mahalanobis_top1.ipynb; jeweils PrototypeCellCNN, initialize_prototypes, Loss und Auswertungsschleife.
- **Q Quadratisches Modell:** derselbe Commit, notebooks/06d_cellcnn_quadratic.ipynb, QuadraticCellCNN, regularized_loss, Trainings- und Halbmaximum-Auswertung.
- **L1 Learnable v1:** Commit 544192d0a4a8bd010c67043ce2d2e3ccf2c65b26, notebooks/06b_cellcnn_mahalanobis_learnable_pooling.ipynb; zugehöriger ausgeführter Snapshot im versionierten 100-Split-Ergebnisordner.

- **L2 Learnable v2:** Commit 1d696d6d28561ccda5e080d8b9cfc3d40122b250, dasselbe Notebook; `PrototypeCellCNN.responses`, `pooled_responses`, `regularized_loss`, `evaluate_loader`, `train_outer_split`, `summarize_split`. Geprüfte Trainingszellen: 1, 2, 5, 6, 7, 8; Funktionsprüfungen in Zelle 10, Phänotypauswertung in 11 und 13.

Die Bonusnotebooks 06a, 06b, 06c und Learnable Pooling sind standardmäßig auf Splits 0–2 eingestellt; 06d auf 0–9. Die dokumentierten 100- und 50-Split-Läufe verwenden separat festgehaltene Umfangs- und Pfadüberschreibungen. Notebookstandard und gemessener Laufumfang sind deshalb auseinanderzuhalten.

Ergebnisartefakte und Provenienz

- **R100, SEL100, A100:** im Learnable-Branch versionierter Ordner `results/tables/task6_learnable_pooling_100/`; Vorhersagen, Metriken, Auswahl, Alpha-Werte, Baseline-Referenz, ausgeführtes Notebook und 100 Checkpoints. `run_scope.json` bindet den Lauf an 544192d und dokumentiert die Wiederverwendung der Splits 0–2.
- **RM:** lokaler GrHa, versioniertes Paket `results/tables/bonus_100/`; `modified_predictions.csv`, `modified_metrics.csv`, `modified_selection.csv`, `modified_frequencies.csv`, Baseline-Vergleich und `run_config.json`. Zugehörige 100 Modellcheckpoints liegen lokal im übergeordneten Tabellenordner.
- **RT, RQ:** gespeicherte Outputs der jeweiligen GitHub-Notebooks und zusätzliche lokale Tabellen `task6_mahalanobis_top1_*` beziehungsweise `task6_quadratic_*`; drei beziehungsweise zehn Checkpoints. Lokale Tabellen sind von den versionierten Notebook-Outputs getrennt zu behandeln.
- **R50:** lokaler Learnable-Worktree, `results/tables/task6_learnable_pooling_geometry_50/`; 50 Checkpoints, Vorhersagen, Kandidatenauswahl, `task6_geometry_effect_comparison.csv`, ausgeführtes Notebook und `run_scope.json`. Die Artefakte sind nicht versioniert. Der Lauf wurde vor dem Ergebniscommit mit noch uncommittierten Änderungen vorbereitet; die Hashes seiner Trainingszellen stimmen mit dem aktuellen v2-Code überein.
- **IG:** lokale Tabellen `task5_paper_groups.csv`, `task5_paper_centroids.csv` und `task5_paper_provenance.json`, bezogen auf die neuere lokale 30-Split-Interpretation.
- **D3:** `src/task23_analysis.py` und lokale `task3_annotations.csv`: markerbasierte Interpretation von Clustern, keine gemessenen Zelltyp-Labels.

Die geprüften Trainingscode-Hashes der Learnable-Checkpoints sind:

v1: e9a38d10ca6cb6efb027b638cf6d3d12bb7af1ec8cb0f1debb4ae4552c2fc53f

v2: 15ea74ebad557a1a3d34efd959643f8199913e002fc28994603eec068f362039

Paper und offizielle Referenz

- **P** Arvaniti und Claassen, *Sensitive detection of rare disease-associated cell subsets via representation learning*, Nature Communications 8, 14825 (2017), DOI 10.1038/ncomms14825. Lokales `CellCNN.pdf`, besonders Methods auf Seite 8; zusätzlich Supplementary Methods.
- **O-NK, O-U, O-M** Offizielles Repository `eiriniar/CellCnn`, Commit 0413a9f49fe0831c8fe3280957fb341f9e028d2d: `cellCnn/examples/NK_cell.ipynb`, `NK_cell_ungated.ipynb`, `cellCnn/model.py::build_model`. Die Referenzen wurden gelesen und mit GitHub abgeglichen, nicht als Hauptpipeline ausgeführt.
- **Aufgabenstellung:** lokales `Group_projects_ssbi_2026.pdf`, Aufgaben 4–6; der dort geforderte fünfseitige Gruppenbericht ist von diesem ausführlichen Analyseanhang zu unterscheiden.

Für diese Analyse tatsächlich ausgeführte Prüfungen

- Alle 20 `gated_alive`-FCS-Dateien gelesen: 3.438.750 Ereignisse, 37 ausgewählte Marker, endliche transformierte Werte; alle 23 Baseline-Eingangshashes geprüft.
- Aus gespeicherten Vorhersagen sämtliche Network-AUCs der sechs aufgeführten Läufe nachgerechnet: 363 Modell/Split-Auswertungen mit 2.178 gespeicherten Donorvorhersagen. Das sind wiederholte Beobachtungen derselben 20 Donoren.
- Frequenz- und pooled-response-Metriken aus den jeweiligen Donortabellen nachgerechnet; Kandidatenvollständigkeit, Sortierregel, Split- und Labelzuordnungen geprüft.
- 263 Prototyp-/Quadrat-Checkpoints auf Trainingscodezuordnung und Übereinstimmung ihrer Vorhersagen mit CSV-Artefakten geprüft; sämtliche 150 Learnable-Checkpoints zusätzlich gegen exportierte Alphas geprüft.
- Direkte Distanzformel gegen die implementierte Matrixform, normalisierte positive Gewichte, ReLU-Membership, Pooling, Permutationsinvarianz, Parameterzahlen und tatsächliche Regularisierungsgradienten an den Originalklassen geprüft. Den vorhandenen kleinen Geometrie-Regularisierungstest separat ausgeführt.
- Für Split 0 den Baseline-Scaler und die Halbmaximum-Schwelle aus Originaldaten rekonstruiert. Für alle sechs Architekturen die fünf Prediction-Bags je sechs Testdonoren erneut inferiert: größte absolute Scoreabweichung $9,30 \cdot 10^{-7}$. Frequenz-/Response-Rekonstruktionen wurden für Baseline, Radius/Mean, Radius/Top1 und beide Learnable-Versionen geprüft; die quadratische Frequenz wurde dabei nicht neu aus sämtlichen Trainingszellen rekonstruiert.
- Den fehlgeschlagenen `SavedCellCNN`-Import im neueren lokalen Stand explizit reproduziert.

Es wurde kein vollständiges Notebooktraining und keine neue wiederholte Cross-Validation durchgeführt. Die hier entworfenen Ablationen und der Refit sind Empfehlungen, keine bereits gemessenen Resultate. Modellcode und Originaldaten wurden nicht verändert.